Lecture 3: Ownership

Mid-series exercises and move semantics

Willem Vanhulle

DevLab Rust 2025

Tuesday November 18, 2025

github.com/wvhulle/rust-course-ghent

1.1. Review exercises

Open the `rust-course-ghent` repo in you editor.

The review exercises are in sub-folder `session-3/examples/`.

Ordered by topic and by coverage in this series:

1. Traits basics and **orphan rule** (important)
2. Generics and supertrait bounds
3. Closures and functional programming (less important)
4. Standard library types and traits
5. Extra's (for prepared students)
   - based on additional questions students
   - useful for student projects

To test your solution (from anywhere): `cargo run --example [FILENAME_WITHOUT_EXT]`

Otherwise, you can do Rustlings exercises or start with your student project.

1.2. Exercises Rustlings

(See https://rustlings.rust-lang.org/setup/)

Review first session:

- Ex 2. Functions
- Ex 4. Primitive types
- Ex 7. Structs
- Ex 8. Enums

Review last session:

- Ex 12. Option
- Ex 15. Traits
- Ex 5. Vectors
- Ex 9. Strings

If you have time:

- Ex 14. Generics
- Ex 11. HashMaps

2.1. Comparison

```rust
struct Key {
    id: u32,
    metadata: Option<String>,
}
impl PartialEq for Key {
    fn eq(&self, other: &Self) -> bool {
        self.id == other.id
    }
}
```

(playground link)

Can equality be implemented between different types?

2.1. Comparison

```
1 struct Key {
2     id: u32,
3     metadata: Option<String>,
4 }
5 impl PartialEq for Key {
6     fn eq(&self, other: &Self) -> bool {
7         self.id == other.id
8     }
9 }
```

(playground link)

Can equality be implemented between different types?

No, both types must be the same or one must implement `PartialEq` for the other type.

2.1. Comparison

```rust
use std::cmp::Ordering;
#[derive(Eq, PartialEq)]
struct Citation {
    author: String,
    year: u32,
}
impl PartialOrd for Citation {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        match self.author.partial_cmp(&other.author) {
            Some(Ordering::Equal) => self.year.partial_cmp(&other.year),
            author_ord => author_ord,
        }
    }
}
```

(playground link)

2.1. Comparison

When two references (not pointers) point to an equal object, are they equal?

2.1. Comparison

When two references (not pointers) point to an equal object, are they equal?

Yes, references implement `PartialEq` by dereferencing both sides and comparing the underlying values.

```
1 fn main() {
2     let a = "Hello";
3     let b = String::from("Hello");
4     assert_eq!(a, b);
5 }
```

(playground link)

2.1. Comparison

```rust
#[derive(Debug, Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}

impl std::ops::Add for Point {
    type Output = Self;

    fn add(self, other: Self) -> Self {
        Self { x: self.x + other.x, y: self.y + other.y }
    }
}
```

(playground link)

What happens if you write Not (`!`) in front of something that is not a boolean?

2.1. Comparison

```rust
#[derive(Debug, Copy, Clone)]
struct Point {
    x: i32,
    y: i32,
}

impl std::ops::Add for Point {
    type Output = Self;

    fn add(self, other: Self) -> Self {
        Self { x: self.x + other.x, y: self.y + other.y }
    }
}
```

(playground link)

What happens if you write Not (`!`) in front of something that is not a boolean?

It flips each bit.

2.1. Comparison

```rust
fn main() {
    let s = String::from("hello");
    let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);
    let one = i16::from(true);
    let bigger = i32::from(123_i16);
    println!("{s}, {addr}, {one}, {bigger}");
}
```

(playground link)

When you have a `From` implementation, do you receive a blanket `Into` implementation?

2.1. Comparison

```rust
fn main() {
    let s = String::from("hello");
    let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);
    let one = i16::from(true);
    let bigger = i32::from(123_i16);
    println!("{s}, {addr}, {one}, {bigger}");
}
```

(playground link)

When you have a `From` implementation, do you receive a blanket `Into` implementation?

Yes.

When you have an `Into` implementation, do you receive a blanket `From` implementation?

2.1. Comparison

```rust
fn main() {
    let s = String::from("hello");
    let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);
    let one = i16::from(true);
    let bigger = i32::from(123_i16);
    println!("{s}, {addr}, {one}, {bigger}");
}
```

(playground link)

When you have a `From` implementation, do you receive a blanket `Into` implementation?

Yes.

When you have an `Into` implementation, do you receive a blanket `From` implementation?

No.

Warning

Always implement `From` for your own types.

2.2. Casting

```rust
fn main() {
    let value: i64 = 1000;
    println!("as u16: {}", value as u16);
    println!("as i16: {}", value as i16);
    println!("as u8: {}", value as u8);
}
```

(playground link)

Warning

Prefer using `From` and `Into` over `as` casting whenever possible.

Which trait should I use when casting is fallible (may go wrong)?

2.2. Casting

```rust
fn main() {
    let value: i64 = 1000;
    println!("as u16: {}", value as u16);
    println!("as i16: {}", value as i16);
    println!("as u8: {}", value as u8);
}
```

(playground link)

Warning

Prefer using `From` and `Into` over `as` casting whenever possible.

Which trait should I use when casting is fallible (may go wrong)?

Use the `TryFrom` and `TryInto` traits.

2.2. Casting

```rust
use std::io::{BufRead, BufReader, Read, Result};

fn count_lines<R: Read>(reader: R) -> usize {
    let buf_reader = BufReader::new(reader);
    buf_reader.lines().count()
}

fn main() -> Result<()> {
    let slice: &[u8] = b"foo\nbar\nbaz\n";
    println!("lines in slice: {}", count_lines(slice));

    let file = std::fs::File::open(std::env::current_exe()?)?;
    println!("lines in file: {}", count_lines(file));
    Ok(())
}
```

(playground link)

2.2. Casting

```rust
use std::io::{Result, Write};

fn log<W: Write>(writer: &mut W, msg: &str) -> Result<()> {
    writer.write_all(msg.as_bytes())?;
    writer.write_all("\n".as_bytes())
}

fn main() -> Result<()> {
    let mut buffer = Vec::new();
    log(&mut buffer, "Hello")?;
    log(&mut buffer, "World")?;
    println!("Logged: {buffer:?}");
    Ok(())
}
```

(playground link)

2.2. Casting

2.2.1. Rustlings

- 23_conversions/

2.2.2. Google

- Read trait: `session-3/tests/s3e1-rot.rs`

2.3. Default

```rust
#[derive(Debug, Default)]
struct Derived {
    x: u32,
    y: String,
    z: Implemented,
}

#[derive(Debug)]
struct Implemented(String);

impl Default for Implemented {
    fn default() -> Self { Self("John Smith".into()) }
}

fn main() {
    let default_struct = Derived::default();
    dbg!(default_struct);

    let almost_default_struct =
        Derived { y: "Y is set!".into(), ..Derived::default() };
    dbg!(almost_default_struct);

    let nothing: Option<Derived> = None;
    dbg!(nothing.unwrap_or_default());
}
```

(playground link)

3.1. Memory layout

```
1  fn main() {
2      let s1 = String::from("Hello");
3  }
```

(playground link)

What data does the `s1` variable contain at runtime?

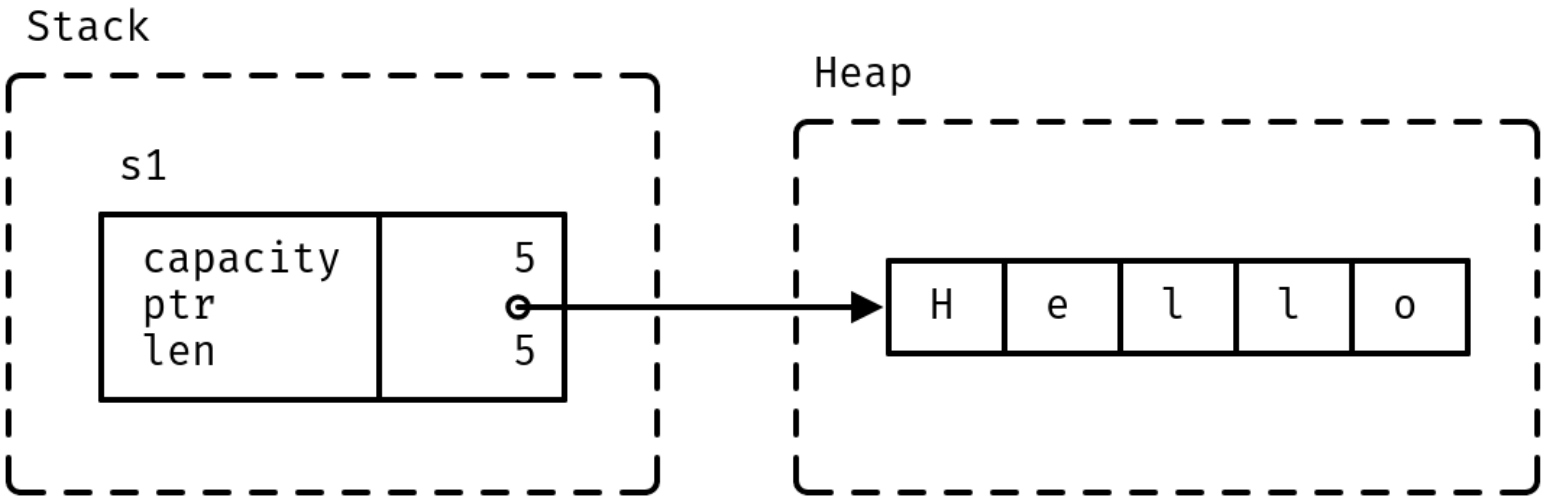
3.1. Memory layout

```
1 fn main() {
2     let s1 = String::from("Hello");
3 }
```

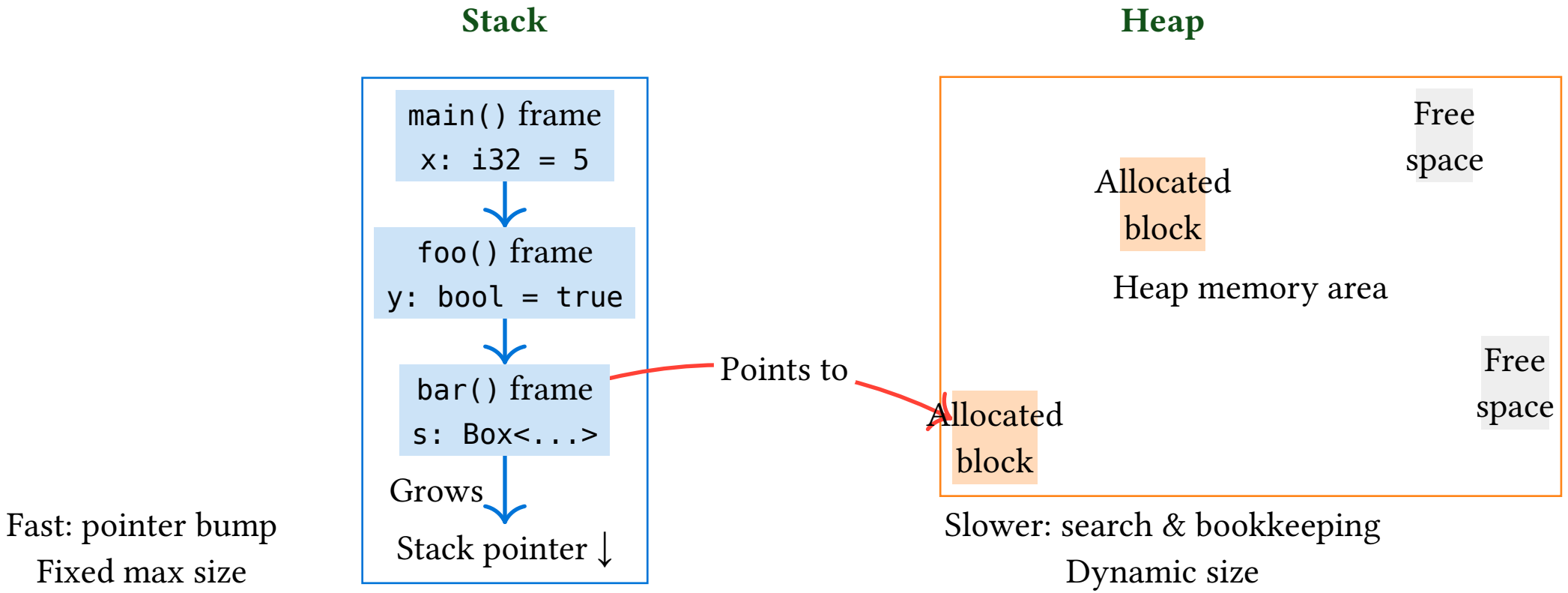
(playground link)

What data does the `s1` variable contain at runtime?

Capacity, address buffer, length.



3.2. Stack and heap



3.2. Stack and heap

```rust
fn main() {
    let mut s1 = String::from("Hello");
    s1.push(' ');
    s1.push_str("world");
    // DON'T DO THIS AT HOME! For educational purposes only.
    // String provides no guarantees about its layout, so this could lead to
    // undefined behavior.
    unsafe {
        let (capacity, ptr, len): (usize, usize, usize) = std::mem::transmute(s1);
        println!("capacity = {capacity}, ptr = {ptr:#x}, len = {len}");
    }
}
```

(playground link)

3.3. Memory challenges

What is the most important challenge when managing memory?

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

Dangling pointers (use after free), double frees, buffer overflows.

What are the two different strategies to avoid memory corruption

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

Dangling pointers (use after free), double frees, buffer overflows.

What are the two different strategies to avoid memory corruption

Automatic or manual memory management.

3.4. Ownership

Every Rust value has precisely one owner at all times.

```
1 struct Point(i32, i32);
2
3 fn main() {
4     {
5         let p = Point(3, 4);
6         dbg!(p.0);
7     }
8     dbg!(p.1);
9 }
```

(playground link)

What does this have in common with automatic garbage collection?

3.4. Ownership

Every Rust value has precisely one owner at all times.

```
1 struct Point(i32, i32);
2
3 fn main() {
4     {
5         let p = Point(3, 4);
6         dbg!(p.0);
7     }
8     dbg!(p.1);
9 }
```

(playground link)

What does this have in common with automatic garbage collection?

Rust's ownership system uses roots as well, but roots are determined statically at compile time.

3.5. Move semantics

```rust
fn main() {
    let s1 = String::from("Hello!");
    let s2 = s1;
    dbg!(s2);
    // dbg!(s1);
}
```

(playground link)

What is the ownership flow?

3.5. Move semantics

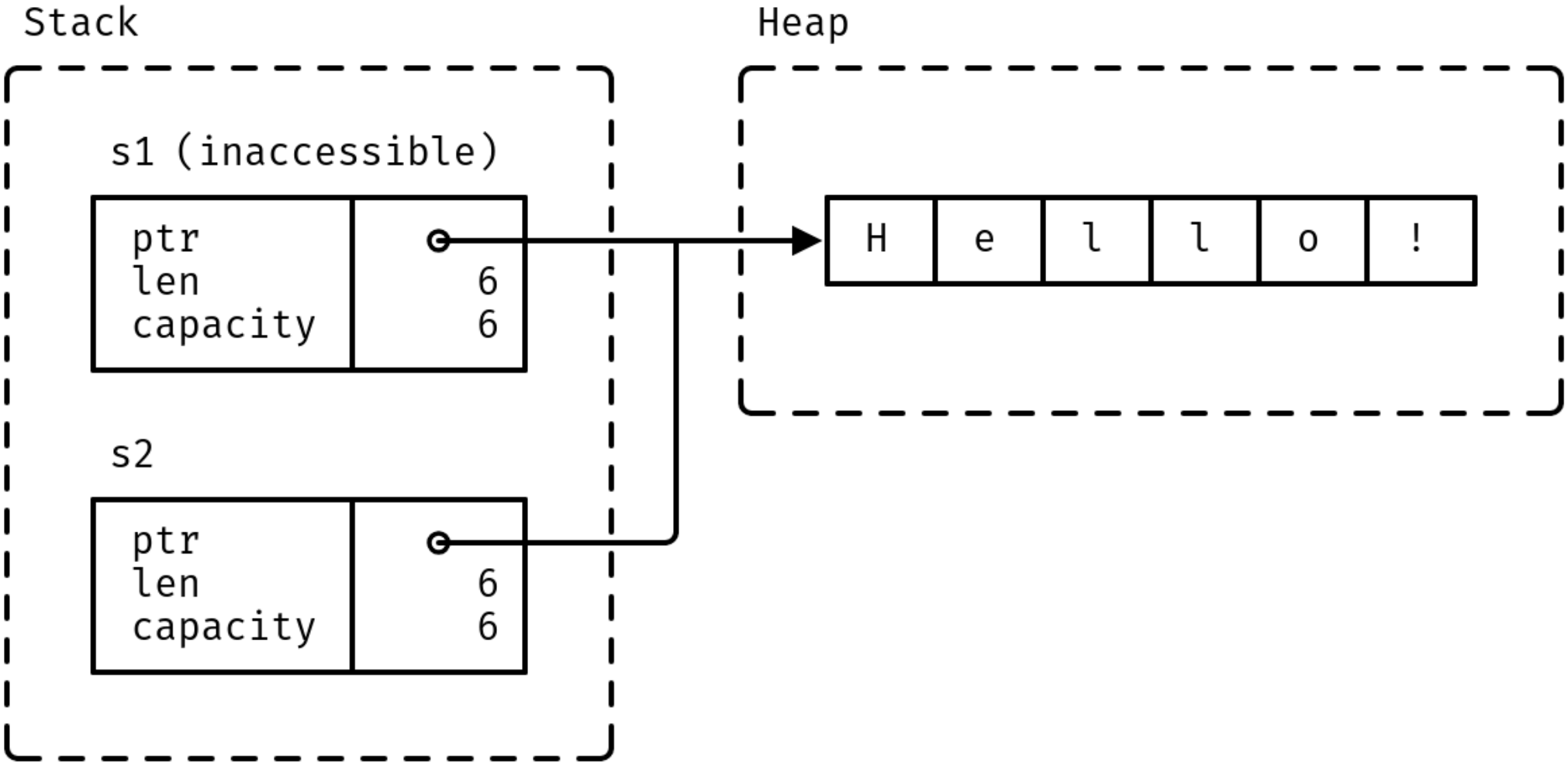
```rust
fn main() {
    let s1 = String::from("Hello!");
    let s2 = s1;
    dbg!(s2);
    // dbg!(s1);
}
```

(playground link)

What is the ownership flow?

`s1` owns the string. When `s1` is assigned to `s2`, ownership moves to `s2`. `s1` is no longer valid.

3.5. Move semantics



3.5. Move semantics

When you pass a value to a function, the value is assigned to the function parameter. This transfers ownership:

```rust
fn say_hello(name: String) {
    println!("Hello {name}")
}

fn main() {
    let name = String::from("Alice");
    say_hello(name);
    // say_hello(name);
}
```

(playground link)

How does this differ from what C++ does?

3.5. Move semantics

When you pass a value to a function, the value is assigned to the function parameter. This transfers ownership:

```rust
fn say_hello(name: String) {
    println!("Hello {name}")
}

fn main() {
    let name = String::from("Alice");
    say_hello(name);
    // say_hello(name);
}
```

(playground link)

How does this differ from what C++ does?

C++ copies the value by default, unless you use references or move semantics explicitly.

3.5. Move semantics

How do you restore C++ behaviour where you copy when necessary?

3.5. Move semantics

How do you restore C++ behaviour where you copy when necessary?

Derive the `Copy` trait.

```
1 fn main() {
2     let x = 5;
3     let y = x;
4     println!("x = {x}, y = {y}");
5 }
```

(playground link)

Are `Strings` in Rust `Copy`? (Do they get cloned automatically on assignment?)

3.5. Move semantics

How do you restore C++ behaviour where you copy when necessary?

Derive the `Copy` trait.

```
1 fn main() {
2     let x = 5;
3     let y = x;
4     println!("x = {x}, y = {y}");
5 }
```

(playground link)

Are `Strings` in Rust `Copy`? (Do they get cloned automatically on assignment?)

No.

3.6. Clone

```rust
fn say_hello(name: String) {
    println!("Hello {name}")
}

fn main() {
    let name = String::from("Alice");
    say_hello(name.clone());
    say_hello(name);
}
```

(playground link)

3.6. Clone

Values which implement Drop can specify code to run when they go out of scope:

```
1 struct Droppable {
2     name: &'static str,
3 }
4
5 impl Drop for Droppable {
6     fn drop(&mut self) {
7         println!("Dropping {}", self.name);
8     }
9 }
```

(playground link)

3.6. Clone

```
1  fn main() {
2      let a = Droppable { name: "a" };
3      {
4          let b = Droppable { name: "b" };
5          {
6              let c = Droppable { name: "c" };
7              let d = Droppable { name: "d" };
8              println!("Exiting innermost block");
9          }
10         println!("Exiting next block");
11     }
12     drop(a);
13     println!("Exiting main");
14 }
```

(playground link)

Explain what will be printed?

3.6. Clone

```
1  fn main() {
2      let a = Droppable { name: "a" };
3      {
4          let b = Droppable { name: "b" };
5          {
6              let c = Droppable { name: "c" };
7              let d = Droppable { name: "d" };
8              println!("Exiting innermost block");
9          }
10         println!("Exiting next block");
11     }
12     drop(a);
13     println!("Exiting main");
14 }
```

(playground link)

Explain what will be printed?

...

3.7. Exercise

3.7.1. Google

- `session-3/examples/s3e4-builder.rs`

3.7.2. Rustlings

- 05_vecs/
- 09_strings/
- 06_move_semantics/